

DESN2000 Notes

Contents

Lecture 1	2
Microprocessor	2
Instructions	2
Data types	2
Processor Modes	2
Program status registers	2
Pipeline	3
Assembly code format	3
Assembler directives	3
AREA	3
EQU	4
ENTRY	4
DCB, DCW, DCD	4
ALIGN	4
SPACE, END	4
Lecture 2	4
ARM data processing instructions	4
ADD / SUB	5
Data move instructions	5
Logical instructions	5
Shifting / rotate	5
Multiplication	6
Load-store architecture	6
Endianness	8

Lecture 1

Microprocessor

Made up of:

- Arithmetic Logic Unit (ALU)
- Registers and internal bus structure
- Control unit (CU) - can be hardwired or microprogrammed

Using the **LCP2478 ARM7TDMI** processor that implements the **ARMv4** instruction set.

TDMI:

- T: thumb (supports 32 bit and 16 bit compressed instruction set)
- D: debug (on chip debug supports, hardware breakpoints)
- M: multiplier (fast / enhanced multiplier)
- I: ICE (Embedded ICE, in-circuit emulation logic for debugging)

Instructions

1st input always from register bank. 2nd input can be:

- Register
- Immediate operand
- A shifted register value

Result goes to a register

e.g

```
ADD R1, R1, #0x5           ; opcode out-reg, in1-reg, in2-immediate
ADD R1, R2, R3, LSL #3     ; opcode out-reg, in1-reg, in2-reg-3-left-shift
```

Data types

bit: 1b byte: 8b halfword: 16b word: 32b

Each instruction is 32b.

Processor Modes

Has seven processor modes, 6 privileged, 1 unprivileged.

Mode	Description	Privileged
Supervisor (SVC)	Entered on reset and when a software interrupt is executed	YES
FIQ	Entered when a high priority interrupt is raised	YES
IRQ	Entered when a low priority (normal) interrupt is raised	YES
Abort	Used to handle memory access violations	YES
Undef	Used to handle undefined instructions	YES
System	Privileged mode using the same registers as User mode	YES
User	Mode under which most applications / OS tasks run	NO

Each mode (except System) has its own set of banked registers so that processor can handle events like interrupts and memory access violations quickly without losing previous state.

Program status registers

31	30	29	28	27:8	7	6	5	4:0
----	----	----	----	------	---	---	---	-----

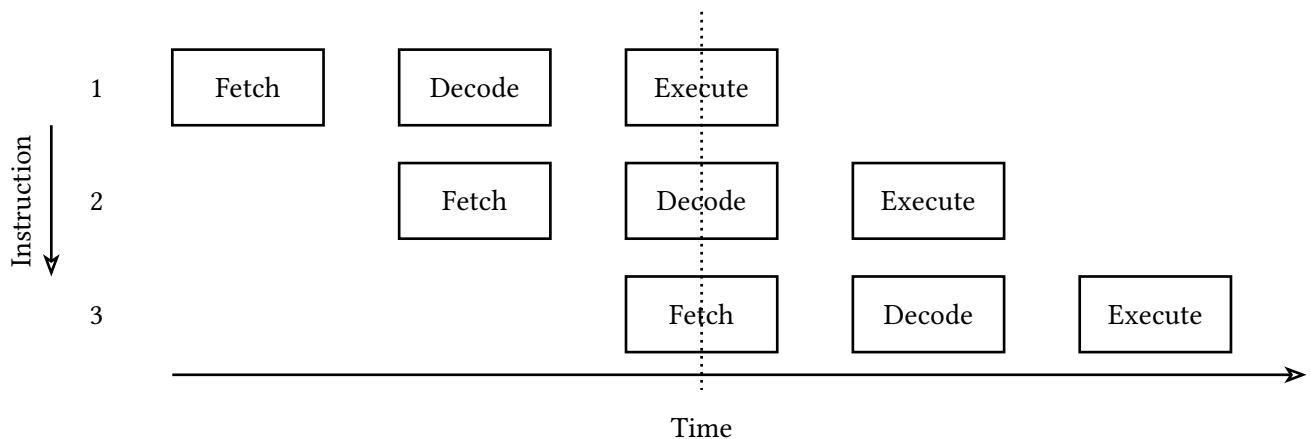
N	Z	C	V	zero	I	F	T	M
---	---	---	---	------	---	---	---	---

- Conditional flag bits:
 - N: negative
 - Z: zero
 - C: carry out
 - V: overflow
- I: disable IRQs
- F: disable FIQs
- T: ARM Thumb instructions (0 for ARM)
- M: Mode bits
 - 10000: User mode
 - 10001: FIQ mode
 - 10010: IRQ mode
 - 10011: Supervisor mode
 - 10111: Abort mode
 - 11011: Undefined mode
 - 11111: System mode

Pipeline

ARM7TDMI has a 3 stage pipeline architecture:

- Fetch
- Decode
- Execute



Assembly code format

<label> <instruction / directive / pseudo-instruction> [; comment]

Label a memory address that can be used like a pointer, is mapped to a physical address by the linker

Instruction an ARM instruction

Directive instructions to the assembler

Pseudo-instruction a fake instruction to ease programming, is converted into real instructions during assembly

Assembler directives

AREA

Defines a block of data or code in the source file.

AREA <section_name> [, attribute]*

If section_name starts with a digit, it must be enclosed in bars: |1_test|. Optional attributes can be:

- CODE a section of code typically declared as read-only
- DATA a section of data, can be declared as read-write
- READONLY declares that a section cannot be modified
- READWRITE declares that a section can be modified

- **ALIGN=expr** aligns the section into a 2^{expr} -byte boundary

EQU

Gives meaningful names to numeric constants in the program, similar to #define.

```
<name> EQU <expr>
```

E.g

```
SRAM_BASE EQU 0x400000000
table EQU label+8
```

ENTRY

Declares an entry point to the program, which must have at least one ENTRY. In projects with multiple source files, each source will have an ENTRY directive.

E.g

```
AREA testprog, CODE, READONLY
ENTRY
```

DCB, DCW, DCD

Used to declare initial run-time content in the memory at byte, halfword and word level.

DCD: Define Constant Byte DCW: Define Constant Half-Word DCD: Define Constant Word

```
[label] DCB expr [, expr]*
```

label: memory address of the starting byte expr: either a numeric expression that evaluates to an integer or a string.

e.g

```
; DCB
C_string DCB "C_string", 0 ; string are not 0 terminated
```

ALIGN

Aligns the current location in memory to a specified boundary by padding with zeros.

```
ALIGN [expr[, offset]]
```

if no arguments, defaults to **ALIGN 4**.

Aligned to the next lowest address of the form: $\text{offset} + n \times \text{expr}$ $n \in \mathbb{Z}$ is automatically selected to minimise padding.

SPACE, END

SPACE allocates a block of memory initialised to zero.

```
<label> SPACE <expr>
```

expr: specifies the number of bytes to reserve as zero.

END declares the end of the source file.

Lecture 2

ARM data processing instructions

- Arithmetic: ADD, SUB, ADC, ...
- Comparison: CMP, CMN, TST, TEQ
- Logical: AND, EOR, ORR, BIC, ...

- Data movement: MOV, MVN

4-field instruction format: <opcode> <dest> <src1> <src2>

- Opcode: name of the operation
- Destination: destination; a register
- src1: 1st source; a register
- src2: 2nd source; a register, shifted register, or an immediate

ADD / SUB

ADD addition <s1> + <s2>

ADDS ADD but saves condition code flags

SUB subtraction <s1> - <s2>

SUBS SUB but saves condition code flags

RSB Reverse subtraction <s2> - <s1>

ADC add with carry

SBC subtract with carry

RSC reverse subtract with carry

e.g.

```
; 64-bit addition
;   R1 R0
; + R3 R2
; -----
;   R5 R4
```

```
ADDS  R4, R0, R2
ADC   R5, R1, R3
```

```
; 64-bit subtraction
;   R1 R0
; - R3 R2
; -----
;   R5 R4
```

```
SUBS  R4, R0, R2
SBC   R5, R1, R3
```

Data move instructions

MOV move <dest> = <s1>

MVN move negative <dest> = -<s1>

Logical instructions

AND bit-wise and

ORR bit-wise or

EOR bit-wise xor

BIC bit clear <dest> = <s1> AND (NOT <s2>)

Shifting / rotate

LSL logical left shift

LSR logical right shift

ASR arithmetic right shift
ROR Rotate right
RRX Rotate right through carry

Multiplication

- Is expensive in clock cycles, power and circuit complexity.
- Multiplication by a constant can be achieved by decomposing number into multiples of powers of two.

E.g.

```
; B = 5 * A = (2^2 + 1)*A = 2^2*A + A
ADD V2, V1, V1, LSL #2
```

```
; B = 105 * A = (15 * 7)A = (2^4 - 1)(2^3 - 1)*A
RSB V2, V1, V1, LSL #4
RSB V2, V2, V2, LSL #3
```

Load-store architecture

Data processing instructions only work on registers. Thus to act on data in memory, it must be loaded into a register first.

Addressing modes

- Pre-indexed ++address
 - offset is added to the base register before the load/store.
 - The load/store takes place at the address pointed by (base + offset)
 - Optionally updates the base register with new address
- Post-indexed address++
 - offset is added to the base after the load/store.
 - The load/store happens at the address pointed by the base register.
 - Always updates the base register with new address.

```
; <opcode>[cond] <rd> \[<rn>, <offset>\][!]  
; opcode  LDR (load word) STR (store word)  
; cond    Optional conditional execution  
; rd      Destination register for LDR and source register for STR  
; rn      Base register  
; offset  Offset from the base register; immediate, register, or shifted register.  
; !       Optionally write effective address back to base register
```

```
LDR V1, [V2, #12] ; load a word into V1 from V2 + 12  
LDR V1, [V2, #12]! ; load a word into V1 from V2 + 12 and store V2 + #12 into V2
```

```
LDR V1, [V2, V3] ; load a word into V1 from V2 + V3  
LDR V1, [V2, V3, LSL #2]! ; load a word into V1 from V2 + V3 << 2 and store V2 + V3<<2 into V2
```

```
STR V1, [V2, #2] ; Store the contents of V1 at V2 + 2  
STR V1, [V2, #12]! ; Store the contents of V1 at V2 + 12 and store V2 + 12 at V2
```

```
STR V1, [V2, V3] ; Store the contents of V1 at V2 + V3  
STR V1, [V2, V3, LSL #2]! ; Store the contents of V1 at V2 + V3<<2 and store V2 + V3<<2 at V2
```

```
; <opcode>{cond} <rd> \[<rn>\], offset  
; opcode  LDR (load word) STR (store word)
```

```

; cond    Optional conditional execution
; rd      Destination / Source register
; rn      Base register
; offset  Added to base

```

```

LDR V1, [V2], #12 ; load a word into V1 from V2 and store V2 + 12 at V2
LDR V1, [V2], V3  ; load a word into V1 from V2 and store V2 + V3 at V2
LDR V1, [V2], V3, LSL #3 ; load a word into V1 from V2 and store V2 + V3<<3 at V2

```

```

STR V1, [V2], #12 ; Store the contents of V1 at V2 and Store V2 + 12 at V2

```

e.g

```

; C = A + B
; Base: 0x4000
; A: offset 0
; B: offset 4
; C: offset 8

```

```

MOV V1, #0x4000
LDR V2, [V1, #0]
LDR V3, [V1, #4]
ADD V2, V2, V3
STR V2, [V1, #8]

```

LDR / STR variants

Load		Store	
LDR	Load a word	STR	Store a word
LDRB	Load a byte	STRB	Store a byte
LDRH	Load a half word	STRH	Store a half word
LDRSB	Load a signed byte		
LDRSH	Load a signed halfword		

Load / Store Multiple:

```

STR a1, [v1], #4
STR a2, [v1], #4
STR a3, [v1], #4
STR a4, [v1], #4

```

=>

```

; Store Multiple Increment After
STMIA v1!, {a1-a4}

```

```

STR a1, [v1, #4]!
STR a2, [v1, #4]!
STR a3, [v1, #4]!
STR a4, [v1, #4]!

```

=>

```

; Store Multiple Increment Before
STMIB v1!, {a1-a4}

```

```

STR a1, [v1]
STR a2, [v1, #4]
STR a3, [v1, #8]

```

```

STR a4, [v1, #12]
=>
; Store Multiple Increment After (no !)
STMIA v1, {a1-a4}

STR a1, [v1, #4]
STR a2, [v1, #8]
STR a3, [v1, #12]
STR a4, [v1, #16]
=>
; Store Multiple Increment Before (no !)
STMIB v1, {a1-a4}

```

Endianness

- Little endian: least significant byte is stored at lowest memory address.
- Big endian: least significant byte is stored at the highest memory address.

ARM is little-endian by default.